💧 Make Commits (Criando versões do seu projeto)

Se antes você estava **preparando**, agora você vai **registrar oficialmente**.

🧠 O que é um commit?

👉 Um **commit** é um ponto salvo no tempo.

É como tirar uma foto do seu código naquele momento.

📸 Analogia forte

- `git add` → você escolhe o que vai na foto
- `git commit` → você tira a foto

⚙️ 1. `git commit` — salvando mudanças

📌 Comando básico:

```
git commit
```

👉 Isso abre um editor para você escrever a mensagem

💡 Forma mais usada:

```
git commit -m "mensagem"
```

🔧 Exemplo real

```
git add app.txt
git commit -m "adiciona arquivo inicial"
```

🧠 O que acontece por trás?

Quando você faz um commit:

- Git salva os arquivos
- cria um ID único
- registra autor, data e mensagem

📝 2. Mensagem de commit (EXTREMAMENTE IMPORTANTE)

👉 Isso separa iniciante de profissional.

❌ Ruim:

```
git commit -m "mudanças"
```

☑ Bom:

```
git commit -m "corrige erro no cálculo de frete"
```

🎯 Regra de ouro:

Se alguém ler seu commit, ele entende o que foi feito.

🧠 Mentalidade de engenheiro

Um commit deve representar:

👉 **UMA ideia única**

💧 Exemplo errado:

```
- corrige bug
- adiciona botão
- muda layout
```

Tudo em um commit ❌

💧 Exemplo certo:

```
commit 1: corrige bug no login
commit 2: adiciona botão de cadastro
commit 3: ajusta layout da tela
```

⚡ 3. `git commit -am` — atalho

📌 Comando:

```
git commit -am "mensagem"
```

🧠 O que faz?

- adiciona arquivos modificados
- faz commit direto

⚠️ Atenção:

👉 NÃO adiciona arquivos novos

🔧 Exemplo

```
git commit -am "corrige erro"
```

Funciona só se o arquivo já estava sendo rastreado.

🔄 4. `git commit --amend` — corrigir commit

📌 Comando:

```
git commit --amend
```

🧠 O que faz?

Permite:

- alterar mensagem
- adicionar arquivos esquecidos

🔧 Exemplo

Você esqueceu um arquivo:

```
git add arquivo.txt
git commit --amend
```

👉 Ele atualiza o commit anterior

⚠️ Cuidado

Não use isso se já fez `git push` (pode causar conflito)

🧬 5. O histórico de commits

Cada commit cria uma linha do tempo:

```
commit 3
↓
commit 2
↓
commit 1
```

🔍 Ver commits:

```
git log
```

ou:

```
git log --oneline
```

🔥 Exemplo completo (vida real)

```
git init

echo "login" > app.txt
git add .
git commit -m "cria sistema de login"

echo "validação" >> app.txt
git add .
git commit -m "adiciona validação de usuário"

echo "bug fix" >> app.txt
git add .
git commit -m "corrige erro na validação"
```

🧠 O que você está fazendo de verdade

Você está:

- criando histórico
- documentando evolução
- organizando seu projeto
- permitindo voltar no tempo

⚠️ Erros comuns de iniciante

❌ 1. Commits gigantes

→ difícil de entender

❌ 2. Mensagens ruins

→ ninguém entende nada

❌ 3. Não fazer commit

→ perde histórico

🎯 Boas práticas profissionais

- commits pequenos
- mensagens claras
- frequência alta
- uma ideia por commit

🚀 Evolução mental

Agora você saiu de:

👉 “salvar arquivos”

Para:

👉 **controlar evolução de software**

🔥 Próximo nível

Agora o Git começa a ficar realmente poderoso:

👉 **Branches (trabalho paralelo sem quebrar o projeto)**